

HOS-LS: 面向 AI 生成代码的「静态规则 + 大模型多智能体」混合漏洞扫描系统

HOS-LS: A Hybrid Static-Analysis and LLM Multi-Agent System for AI-Generated Code Security Scanning

作者: 钱佳宏 导师: _ _ _ 日期: 2026-08 | 论文初稿 v2 (标准版) · 中文草稿, 术语保留英文, 投稿前整体转英文

摘要

AI 代码生成的普及显著提高了开发效率, 也放大了代码安全审查的压力: 传统静态分析规则覆盖不全、误报漏报双高; 纯大语言模型 (LLM) 检测误报率高且判定不可解释; 2026 年出现的多智能体漏洞检测系统多依赖模糊测试复现或 LLM 自评完成验证, 指标口径混乱、漏洞声称难以核验。本文提出 **HOS-LS**——面向 AI 生成代码的「静态规则 + 大模型多智能体」混合漏洞扫描系统 (v0.3.3.17, Python, AGPLv3), 采用四阶段分层架构: 静态规则层快速召回候选, Search Agent 评分筛选 Top-K, 7 个 LLM Agent 深度分析, 最后由 DynamicLoader 与 Validator Registry 等确定性执行器完成 Exploit 生成与验证, 验证不依赖 LLM 自评。在 RepoPairBench (100 个 Python 漏洞-修复对, 均挂 2021–2025 CVE) 上的实测结果表明: 静态规则层召回 76.0% (误报 76.0%, 定位为候选生成器); AI 层双口径检出率为 **CONFIRMED 35.0%、高危识别 49.0%**; 完整源码与跨函数上下文注入可将代表性样本检出由 0/4 提升至 3/4。在 10 样本与裸 LLM 的同台受控对比中 (同模型 deepseek-v4-flash), HOS-LS 检出率达 **80.0%** (识别口径 8/10; 严格 CONFIRMED 口径 70.0%), 而误报由裸 LLM 的 90.0% 降至 30.0%。与 FuzzingBrain V2、AEGIS、DREA 等系统的对比分析表明, HOS-LS 在验证确定性、口径透明与可核验性上具有结构性优势。

关键词: AI 生成代码安全; 漏洞检测; 多智能体; 大语言模型; 静态分析; 确定性验证

Abstract: The proliferation of AI-generated code has intensified the demand for security review, yet existing approaches suffer from incomplete static-analysis rules, unexplainable LLM-based detection, and non-reproducible claims in 2026 multi-agent systems. This paper presents HOS-LS, a hybrid static-analysis and LLM multi-agent scanner with a four-stage architecture: static rule screening, Search-Agent Top-K selection, seven-Agent deep analysis, and deterministic exploit generation and verification. On RepoPairBench (100 Python vulnerable-fixed pairs with 2021–2025 CVEs), the static layer achieves 76.0% recall, and the AI layer reaches 35.0% (CONFIRMED) / 49.0% (recognition) detection under dual criteria; injecting full source and cross-function context raises a representative subset from 0/4 to 3/4. Compared with FuzzingBrain V2, AEGIS, and DREA, HOS-LS offers structural advantages in verification determinism and metric transparency.

Keywords: AI-generated code security; vulnerability detection; multi-agent systems; large language models; static analysis; deterministic verification

1 引言

1.1 研究背景与动机

AI 辅助编程工具已深度融入软件开发流程, 但其生成代码的安全性长期存疑, 相关风险已成为领域研究焦点[14][15], 安全审查需求由此激增。与此同时, 对 2026 年主流多智能体漏洞检测论文的系统分析显示, 这些工作普遍存在三类问题: 指标口径不完整、漏洞声称不可核验、验证依赖 LLM 自评。漏洞检测系统的**可信度问题**已成为领域公认痛点, 也为本工作提供了明确的问题定位。

1.2 现有方法的不足

传统静态分析与深度学习检测。

静态分析规则依赖专家手工维护, 覆盖不全, 跨函数、跨文件漏洞漏报率高, 误报漏报双高 (Johnson 2013 / Habib 2018) [22]; 以 VulDeePecker [4]、Devign [5]、CodeBERT [6] 为代表的深度学习检测止于函数级分类, 且训练数据存在严重泄漏——PrimeVul [7] 指出 BigVul 等基准的泄漏率最高达 18.9%, 曾被广泛引用的函数级 F1 68.26% 在严格去重与时间切分后仅剩 3.09%。

纯 LLM 直接检测。 无工具、无上下文，准确率接近随机且判定不可解释 (Ullah et al., S&P 2024) [9]; Sifting-the-Noise (ISSTA 2026) [10] 进一步表明，LLM Agent 在过滤静态分析误报的同时会吞掉 22.25% 的真实漏洞。

2026 年多智能体系统。 FuzzingBrain V2 [25] (LLM + fuzzing, 限 C/C++ 崩溃型)、AEGIS [26] (LLM + CPG)、DREA [27] (双 Agent 解耦) 等系统的验证多依赖 fuzzing 复现或 LLM 自评，且指标口径存在可复现性问题：FzB 声称的零日数量 29 与 41 相互矛盾，90% 检出为赛后回测结果；AEGIS 的 FPR 采用非常规口径 ($fp = P-V + P-R$)，且 Audit 与 Verifier 使用同一模型，存在自证偏置。

1.3 本文贡献

针对上述不足，本文提出 **HOS-LS——「规则保下限、LLM 提上限、确定性验证兜底」** 的混合扫描系统，并把口径公开作为系统默认能力。主要贡献如下：

1. **C1 四阶段混合架构**：静态规则 → Search Agent Top-K 筛选 → LLM 深度分析 → Exploit 生成与确定性验证，是「LLM 假设 + 确定性验证」范式的工程化落地，分层扫描将深度分析严格限制在候选子集，显著降低大模型调用成本（实测成本数据见 §4.3）；
2. **C2 多智能体编排与信号状态机**：7 个 Agent 由 LangGraph 编排，风险信号经 NEW → REFINED → ACCEPTED/REJECTED 状态机流转，Agent-3 验证覆盖度不足时强制切换串行，CONFIRMED 优先于 REJECTED，回答「多 Agent 如何协作才有效」；
3. **C3 hybrid 必要性与代价实证**：以分层、子集、上下文与优化历程四组对比数据量化各机制层的贡献与成本，支持「机制贡献 ≠ 模型贡献」；
4. **C4 评测口径自我审计**：全部指标公开定义与分子分母，漏洞声称挂 CVE/样本 ID，评测数据与脚本随论文公开，预印本引用注明状态。

2 相关工作

2.1 深度学习漏洞检测

VulDeePecker [4] 首次将深度学习引入漏洞检测，将问题建模为函数级分类；Devign [5] 以图神经网络学习程序语义；CodeBERT [6] 等预训练代码模型提供了通用代码表示。这类方法的问题在于：以函数级切片为单元，缺乏跨函数、跨文件上下文；训练基准存在泄漏，指标被系统性高估 (PrimeVul [7])。HOS-LS 与之的差异在于以仓库级上下文、多智能体推理与确定性验证构成新的检测范式。

2.2 大语言模型漏洞检测与混合方法

IRIS [16] 用 LLM 过滤静态分析告警，是「规则 + LLM」两段式混合的代表；Vul-RAG [17] 以知识级 RAG 增强 LLM 检测；LLM4Vuln [19] 提出解耦 LLM 推理能力的评测框架；LLMxCPG [18] 以代码属性图引导 LLM。这类方法的共同局限是缺少确定性验证闭环，混合停留在「规则过滤 + LLM 判定」两段。HOS-LS 的混合为「规则 → LLM → 确定性验证」三段闭环，验证环节不依赖 LLM。

2.3 大语言模型多智能体漏洞检测

FuzzingBrain V2 [25]、AEGIS [26]、DREA [27] 以及 AutoTrace、Revelio、CLEAR、VulAgentRL、ALIBI [30–34] 构成了 2026 年多智能体漏洞检测的主流图景，系统级对比如表 1 所示。FuzzingBrain V2 以 OSS-Fuzz 复现作为验证标准，但限于 C/C++ 崩溃型漏洞；AEGIS 以 CPG 图谱驱动四层推理管线，在 PrimeVul 配对样本上首次突破 100 对，但其 FPR 口径非常规且审计与验证环节同模型；DREA 以「推理/探索」双 Agent 解耦降低 token 成本，但其基准样本存在测试文件污染，判定依赖 LLM 裁判。HOS-LS 的差异化在于验证由确定性执行器完成、全部指标公开定义。

表 1 2026 年多智能体漏洞检测系统对比

维度	FuzzingBrain V2 [25]	AEGIS [26]	DREA [27]	HOS-LS
---	---	---	---	---
技术路线	LLM + fuzzing (动态验证)	LLM + CPG 图谱 (静态推理)	推理/探索双 Agent 解耦	静态规则 → LLM → 确定性验证
验证机制	OSS-Fuzz 复现 (须 fuzzer 可复现)	Audit 单向否决 (与 Verifier 同模型)	LLM 裁判 (judge 自评)	DynamicLoader + Validator Registry + sanitizer 仲裁
Agent 架构	BaseAgent loop + 固定流水线	四层管线 (线索→证据→辩证→元审计)	双 Agent (Reasoning/Exploring)	7 Agent + LangGraph + 信号状态机
知识/检索	fuzz 靶标驱动	CPG 图谱	仓库级上下文	RAG (BM25 + 向量) + NVD/ExploitDB
评测集	AlxCC AFC (C/C++) + 零日	PrimeVul (函数级配对样本)	RepoPairBench (仓库级)	RepoPairBench (当前) + PrimeVul (待补)
代表性口径问题	29 vs 41 零日矛盾; 90% 赛后回测	FPR 口径逆向; Audit/Verifier 同模型	样本修在测试文件; 训练记忆污染	公开全部口径与样本 ID

2.4 漏洞检测评测基准与渗透测试

PrimeVul [7] 以严格去重与时间切分成为函数级评测的事实标准，但其函数级评估存在跨函数盲区且未对比 SAST 基线——「混合系统相对传统 SAST 的实用增量」正是本文的立足点。ReposVul [28] 提供仓库级高质量数据集；Sifting-the-Noise [10] 提供 LLM Agent 过滤误报的对比协议；Risse 等 [24] 对漏洞检测基准的可信度提出系统性批评。渗透测试一侧，PentestGPT [8] 证明 LLM 具备漏洞利用链推理能力，但存在 human-in-the-loop 冒充自动化与训练污染问题——HOS-LS 的 Exploit 自动生成与确定性验证在设计上即针对这两点。

2.5 引用交集说明

与 FuzzingBrain V2 共享约 15–17 条核心引用 (PrimeVul、VulDeePecker、IRIS、Vul-RAG 等)，引用网络落在领域主流通路、无引用孤岛；同时补引其未覆盖的 2026 年多智能体系统与评测协议文献，保证 related work 的覆盖无死角。

3 系统设计

3.1 系统总览

HOS-LS (v0.3.3.17) 面向 AI 生成代码安全扫描，支持 Python/JavaScript/TypeScript/Java/C/C++/Go/Rust 多语言与 6 种扫描模式，提供 Pure-AI (轻量，仅需 AI API) 与完整版 (含 RAG 知识库与 Neo4j 攻击图) 两种运行形态。系统设计重心在工程机制而非模型选型——模型仅作为组件出现，本文全部实验统一采用 DeepSeek-v4-flash，不将模型选型作为卖点。整体架构遵循「候选生成 → 筛选 → 深度分析 → 确定性验证」的漏斗式流水线，各环节职责与成本特征见表 2。

表 2 四阶段分层扫描架构

阶段	职责	主要输出	成本特征
---	---	---	---
Stage 1 静态规则	危险模式全量快速扫描	候选漏洞点	快、全量覆盖
Stage 2 Search Agent	评分筛选 Top-K 文件	高嫌疑文件/函数	低 (Merkle Tree 增量索引)
Stage 3 AI 深度分析	7 Agent 语义分析	漏洞验证结果	仅分析候选子集，显著降本 (实测见 §4.3)
Stage 4 Exploit + 验证	PoC 生成 + 确定性执行	有效 / 误报 / 需复核	确定性执行，不依赖 LLM 自评

3.2 四阶段分层扫描

Stage 1 静态规则引擎对全量文件快速扫描危险函数模式，生成候选漏洞点；Stage 2 Search Agent 按评分公式 $\text{keyword} \times 0.3 + \text{call_chain} \times 0.25 + \text{historical} \times 0.2 + \text{file_type} \times 0.15 + \text{diff} \times 0.1$

对候选文件排序，取 Top-K 进入深度分析，配合 Merkle Tree 增量索引只索引变化文件；Stage 3 仅对候选执行多智能体深度分析；Stage 4 对确认漏洞自动生成 Exploit，由确定性验证器执行并标记「有效 / 误报 / 需复核」。

3.3 Search Agent 候选筛选与混合检索

Search Agent 基于向量嵌入检索相关代码并计算候选评分，仅分析评分最高的 Top-K 文件，显著减少送入 LLM 的代码量。知识层采用混合检索 RAG (BM25 + 向量, PostgreSQL + FAISS)，并支持 NVD 与 ExploitDB 双路径导入：ETL 批量入库 (SQLite) 支撑 CVE/CWE 精确匹配，RAG 向量化支撑语义增强与漏洞知识问答。

3.4 多智能体编排与信号状态机

7 个 Agent 的职责见表 3，由 LangGraph 编排执行。风险信号经状态机 NEW → REFINED → ACCEPTED/REJECTED 流转；Agent-3 验证覆盖度不足 50% 时强制切换串行处理，保证验证完整性；CONFIRMED 结果优先于 Agent-6 的 REJECTED 决策，避免「一票否决」式误杀；每个 Agent 执行后记录 token 用量，支撑成本量化。

表 3 7 个 Agent 的职责

Agent	职责
---	---
0 上下文构建	构建代码上下文、分析文件依赖
1 代码理解	深度理解代码逻辑、分析数据流
2 风险枚举	枚举潜在风险点、生成风险信号
3 漏洞验证	代码级验证、判断攻击路径可行性
4 攻击链分析	分析漏洞关联、构建攻击路径
5 对抗验证	对抗性测试、验证漏洞可利用性
6 最终裁决	综合决策、确定漏洞最终状态

3.5 确定性验证闭环

DynamicLoader 从验证器目录动态加载验证器；AIPOCGenerator 自动生成泛化 PoC；Validator Registry 提供五类验证器 (SQL 注入 / 认证绕过 / SSRF / 凭证泄露 / 反序列化) 并支持热更新；sanitizer 仲裁后输出验证结论；hos-ls verify 支持对历史扫描报告复核。该闭环将验证从「LLM 自评」迁移到「确定性执行器」，是 HOS-LS 可信度的根基，也是对抗 2026 年同类系统「LLM 自己验证自己」批评的核心手段。

3.6 误报控制机制

ContextAnalyzer (服务层调用链追踪)、TypeChecker (参数类型上下文，如 Integer 在 LIMIT 中安全)、InputTracer (用户输入传播路径)、FrameworkSecurityModel (MyBatis-Plus / Spring 框架安全封装语义) 四组件协同控制误报，规则引擎覆盖 OWASP Top 10 全类别。

4 实验评估

4.1 评测协议

采用 RepoPairBench (DREA 自建，100 个 Python 漏洞-修复对，均带 2021–2025 CVE 编号与 commit hash)。样本构造：提取漏洞函数 (code_before) 与修复函数 (code_after)，逐文件扫描；检出判定：静态层存在告警项 (finding > 0)，AI 层风险状态为 CONFIRMED；误报判定：修复后文件仍有告警项。局限声明：单函数样本、无仓库上下文 (与 DREA 仓库级设定有差距)。被测系统为 v0.3.3.17 + 本地修复 (PR #21/#26/#27/#28) + OPT 轮改动 (OPT-P2/OPT-TOKEN)，模型为 deepseek-v4-flash。协议说明：AI 层采用 --pure-ai 模式逐文件扫描。OPT-SAST 改造后，pure-ai 模式下批量分析前先运行 SAST 深度前置过滤 (§4.3(vii))：默认软门控 (证据注入 + 早停，保留盲区检出)，硬门控变体零命中文件完全跳过 AI；本表 AI 层数字基于软门控 (全部文件均经 AI 分析，属 AI 层成本上界)。生产模式 (非 pure-ai) 下 Stage 1 另运行外部 SAST 工具链 (Semgrep/Trivy/Gitleaks) 前置过滤。

4.2 全量评测结果

表 4 给出 100 pairs 全量结果。静态规则层对漏洞文件召回 76.0%、对修复文件误报 76.0%——按 200 个文件样本计算，其 Precision = 76/(76+76) = **50.0%**；pair-wise 双端正确率（漏洞检出且修复端不报）仅 4.0%（100 对中仅 4 对），说明静态层是**候选生成器而非判定器**：只匹配危险模式、无法判别可利用性。这一事实正是四阶段架构的设计动机（规则层保召回，由 AI 与验证层收敛误报），也印证了 Sifting-the-Noise 的研究主题[10]。AI 层双口径的语义如下：**CONFIRMED 口径（35.0%）反映系统「确定性验证」能力**——Agent 识别 + 验证链完整（PoC 生成/数据流/攻击链关联）后才计入；**识别口径（49.0%）反映「语义理解深度」**——Agent 已识别出真实高危（22 个 WEAK/UNCERTAIN 样本经复核几乎全部是真实漏洞：os.system 命令注入、Jinja2 无沙箱渲染、不安全 yaml.load、URL 路径拼接等），仅因验证链未走完而未达 CONFIRMED。两者不是软硬指标的替换，而是两个不同维度的能力指标，因此本文采用双口径报告。

表 4 RepoPairBench 100 pairs 全量结果 (deepseek-v4-flash)

层 / 口径	检出 (vuln)	误报 (patched)	说明
---	---	---	---
静态规则层 (n=100)	76/100 = 76.0%	76/100 = 76.0%	候选生成器：危险模式匹配，不判可利用性 (Precision 50.0%)
AI 层 · CONFIRMED (n=100)	35/100 = 35.0%	—	验证链完整才计入
AI 层 · 识别口径 (n=100)	49/100 = 49.0%	—	含 WEAK/UNCERTAIN 中已识别的真实高危
分层 · 函数内 (55)	43.6%	—	CONFIRMED 口径
分层 · 跨函数 (35)	37.1%	—	CONFIRMED 口径
分层 · 测试文件 (10)	10.0%	—	漏洞在库代码，函数级不可检出
OPT 配置 · CONFIRMED (n=21 受控子集)	10/21 = 47.6%	—	优化配置 (Agent-4 修复 + 输入压缩)，预算受限截断，2026-08-15
OPT 配置 · 识别口径 (n=21)	10/21 = 47.6%	—	同上子集

注：分层数字为 CONFIRMED 口径下 PR #26 修复后全量统计 (v2, 38/100)；AI 层双口径为 PR #28 后全量 (v3, 35/49)。静态层按 200 文件样本口径：Precision 50.0%、Accuracy 50.0%、FPR 76.0%；pair-wise 双端正确率 4.0%。OPT 行为预算受限的受控子集 (n=21, 按文件名序截断)，与 v3 全量不直接可比；同 10 样本单轮受控对比 (表 8 协议) 下，OPT 代码由 v3 单轮 5/10 提升至 6/10。

4.3 分层与子集对比分析

(i) **按样本类型分层** (表 4 末三行)：函数内 43.6%、跨函数 37.1%、测试文件 10.0%——测试文件样本的漏洞位于库代码，函数级切片不可检出。(ii) **误报收敛对比** (表 5)：10 样本子集中静态层产生 155 条告警项 (无差别报警)，AI 层仅 9 条 (高危 5 / 中危 3 / 低危 1)，且与 RepoPairBench 真实 CVE 对应 (如 03e97308 的 statusfile 字段注入、06fdf927 的 XSS)，误报大幅收敛。(iii) **判定明细** (表 6)：10 样本中真阳性 3、修复不完整误报 2、类型错位 2、漏检 3——漏检全部归因于样本特性 (测试文件脏样本、跨函数 XSS) 或描述-类型对齐问题，而非系统缺陷。(iv) **上下文增益** (表 7)：注入完整函数源码与跨文件被调函数后，octoprint 4 个样本由 0/4 提升至 3/4，且注入为空的 2 个样本亦因源码完整性而检出——表明评测集的单函数切片截断了可判定信息，与 Li 2025「上下文对检测至关重要」[23] 的结论一致。(v) **分层筛选的成本必要性 (实测)**：静态规则层 100 文件全量扫描耗时 534s (约 5.3s/文件，无 API 成本)；AI 层单文件完整执行约 130–160s (10 样本两轮实测：5 文件组分别 699s / 755s)，约为静态层的 25–30 倍。四阶段架构通过静态层召回 + Search Agent Top-K 将深度分析限制在候选子集，避免对全部文件执行昂贵的多智能体分析；端到端 token 消耗的精确计量 (含「全量无分层」对照) 列入 §4.7 待补。(vi) **静态门控成本账 (已有数据测算，零新增 API 调用)**：静态层对 100 文件命中 76/100、0 召回 24 个。若采用**硬门控** (AI 仅分析静态命中文件)，按 OPT 截断子集 (n=21) 的 AI 层 token 实测，AI 层 token 可再省 **21.8%**；但代价是丢失 AI 可检出的静态盲区样本——21 子集中 06fdf927 (跨函数 XSS, CWE-79) 即 AI-CONFIRMED 而静态 0 召回，硬门控将漏掉这类「规则看不见但 LLM 能检出」的漏洞。因此系统采用**软门控 + 早停**：静态层保召回，AI 层内 Agent-2 零风险且静态门零命中时提前跳过 Agent-3~6 (实测低 token 样本即早停路径)，在保留盲区检出能力的同时控制成本。(vii) **SAST 深度前置过滤 · 三级 cascade (OPT-SASTR2, 本工作新增)**：pure-ai 模式不再绕过静态过滤，默认流水线为 semgrep/bandit

快扫 (S1) → CodeQL 深扫 (S2) → pure-AI 盲区 (S3) : S1 用社区规则集 (semgrep 337 条 python 规则; 沙箱受限环境自动降级 bandit) 全量快扫产出候选; S2 用 CodeQL (仓库级污点分析底座, python-security-and-quality 官方套件) 对候选深扫, **CodeQL 确认的文件直接产出硬 findings (source=codeql, 0 AI token)** ; S3 只对 S1∪S2 未决文件 (候选验证 + 盲区) 运行 AI。工具链收敛于项目内独立环境 envs/ (sast-venv: semgrep 1.159.0/bandit 1.9.4; codeql 2.26.3 + python-queries 1.8.8; semgrep-rules 337 条), 依赖分组写入 requirements.txt/requirements-sast.txt。**21 子集实测 (松散函数切片) :** CodeQL 确认 0/21 (污点分析需要真实代码结构, 价值在仓库级)、bandit 候选命中 5/21 (193c77fa/1a1914c0 等, 与 AI 检出正相关——S1 提议、AI 确认的 SAST-Genius 模式)、AI 层仍覆盖 21/21 (盲区 06fdf927/08926a1a 保留, 检出不倒退)。**仓库级才是 token 节省的主场:** CodeQL 对真实仓库能确认的漏洞集将直接从 AI 工作集剔除 (免 AI token), 配合第 4 阶段确定性验证, 构成「硬检测定论 + AI 补盲」的成本结构; 仓库级 cascade 评测列入 §4.7 待补 (需完整权限建库)。

表 5 静态层与 AI 层误报收敛对比

层	样本数	告警项数	严重度分布 / 说明
---	---	---	---
静态规则层	100 文件	155	无差别报警 (不区分修复前后)
AI 层 (7 Agent)	10 文件 (子集)	9	高危 5 / 中危 3 / 低危 1, 与真实 CVE 对应

注: 两行文件集不同 (静态为全量 100、AI 为 10 子集), 仅用于展示数量级上的误报收敛趋势; 同一样本的严格对照需仓库级评测。

表 6 AI 层 10 样本判定明细 (按目标 CWE)

样本 (pair)	目标 CWE	AI 判定	类别
---	---	---	---
08926a1a	CWE-22/23	检出 (TP)	真阳性
0a06f338	CWE-601	检出 (TP)	真阳性
0c8d2aef	CWE-400	检出 (TP)	真阳性
0294b9f1	CWE-611	patched 仍报同类 XXE	修复不完整 (误报)
0f0215bf	CWE-918	patched 仍报同类 SSRF	修复不完整 (误报)
00c73b6e	CWE-22	报「硬编码路径」≠路径遍历	类型错位
03e97308	CWE-306	报「注入」≠认证缺失	类型错位
06fdf927	CWE-434/79	跨函数 XSS, 函数级不可见	漏检 (样本特性)
0bb1aef8	CWE-20	漏洞在库代码, 测试文件无漏洞	漏检 (脏样本)
0dc2e99d	CWE-20	描述错位, 关键词未匹配	漏检 (对齐问题)

注: 本表为 10 样本子集的判定质量分析 (早于 PR #28 全量改造), 用于漏检归因; 检出率以表 4 全量双口径为准。

表 7 上下文增益对比 (octoprint 4 样本)

样本 (pair)	CVE	单函数 (code_before 切片)	深 CPG 上下文 (完整源码 + 跨函数注入)
---	---	---	---
6e68fb86	CVE-2022-2822	0	2 (检出)
8c57d15f	CVE-2022-2930	0	2 (检出)
d873de7f	CVE-2022-1430	0	0
ee76e0c9	CVE-2022-3068	0	1 (检出)
合计	—	0/4	3/4

注: n=4 样本量小 (随机性 ±2) ; 注入为空的 2 个样本亦检出, 说明完整函数源码 (相对评测集切片) 是主要变量, 跨文件注入为次要因素。

4.4 与基线工具的同台受控对比 (10 样本)

为回答「多智能体架构相对裸 LLM 与两段式混合的增量」, 在相同 10 个漏洞-修复对上以同一模型 (deepseek-v4-flash) 同台对比三类 AI 工具: ① 裸 LLM 零样本直接判定; ② 静态规则 + LLM 两段式判定 (IRIS 式[16]: 仅对静态规则召回的可疑点做 LLM 判定) ; ③ HOS-LS 7-Agent 完整管线。由于 deepseek-v4-flash 为推理模型、单次运行存在波动 (同一文件多次运行检出 2→1→0), HOS-LS 采用**多轮运行取并集**的口径 (漏检样本最多重跑 3 轮), 并按

CONFIRMED（严格）与「CONFIRMED 或 high/critical」（识别）双口径报告。结果见表 8。

表 8 10 样本三工具同台受控对比 (n=10 pairs, deepseek-v4-flash, 2026-08-15 实测)

工具	检出 (vuln)	误报 (patched)	说明
---	---	---	---
裸 LLM 零样本	9/10	9/10	无工具无上下文，近乎无差别报警
静态规则 + LLM 判定	5/10	3/10	IRIS 式两段：精度尚可但召回不足
HOS-LS · CONFIRMED（多轮并集）	7/10	3/10	验证链完整才计入（严格口径 70.0%）
HOS-LS · 识别口径（多轮并集）	8/10 = 80.0%	3/10	CONFIRMED 或 high/critical 告警项

分析：(i) 裸 LLM 的 9/10 检出以 9/10 误报为代价——它无法区分漏洞代码与修复代码，工程上不可用；HOS-LS 在识别口径 80.0%（仅差 1 个样本）下将误报压缩三分之二，且每个检出都附带 Agent 级证据链。(ii) HOS-LS 的 3 个 patched「误报」经复核均为 RepoPairBench 修复不完整的真实残留风险（00c73b6e 硬编码路径、03e97308 statusfile 仍直拼 sudo rm、0f0215bf SSRF 未彻底收敛），而非「修复后仍报警」式误报——DREA 论文亦声明 patched 仅针对特定 CVE 修复、不代表完全安全。(iii) HOS-LS 漏检 2/10（0bb1aef8 测试文件脏样本、0dc2e99d 描述错位）与全量评测的归因一致（表 6）；0294b9f1（SVG XML 注入，CWE-611）三轮运行均未达 CONFIRMED（前两轮产出 high 级告警项但停在 REFINED），是「识别成功但验证链未走完」的典型样例，恰是识别口径存在意义的证据。(iv) 两段式混合（规则 + LLM）召回仅 5/10，其漏检与静态规则盲区一致（06fdf927、08926a1a、0c8d2aef 等静态 0 召回样本）——说明「规则召回 + LLM 判定」不足以覆盖语义漏洞，需要 HOS-LS 的候选扩检与多智能体验证链条。

4.5 优化历程与机制贡献

表 9 汇总了检出率优化历程。关键结论：Agent-6 判定规则修复（PR #26）使 CONFIRMED 检出由 19.0% 提升至 38.0%（翻倍）；Agent-2 候选风险放宽（PR #27）使无告警样本由 0/6 提升至 3/6；「先分析后判定」改造（PR #28）收敛为双口径 35.0% / 49.0%（CONFIRMED 口径与 v2 持平，识别口径新增 49.0%）。控制实验表明，禁用 NVD 数据库后 10 样本检出 7/10 与启用时相同，排除环境因素。2026-08 的 OPT 轮（本工作新增）：Agent-4 攻击链输出契约放宽 + 确定性兜底单步链（OPT-P2）使 10 样本单轮 CONFIRMED 由 v3 的 5/10 提升至 6/10，且诊断日志确认不再出现「攻击链为空 → Agent-5/6 弱输入」断链；Agent-5/6 上游输入压缩（OPT-TOKEN）使同样本 token 消耗由 769,612 降至 650,140（-15.5%）；确定性升级（OPT-P1/P3）在 RepoPairBench 函数级样本上未触发（Agent-6 输出的 WEAK 高危多对应 Agent-3 REFINED 而非 CONFIRMED，保持严格口径不模糊 CONFIRMED 语义）。这些机制级改动在同一模型下带来检出提升与 token 下降，直接支持「机制贡献 ≠ 模型贡献」：工程机制的优化即可带来显著增益。

表 9 AI 层检出率优化历程（同模型 deepseek-v4-flash）

阶段 / PR	机制改动	CONFIRMED 检出	说明
---	---	---	---
基线 (v1 全量)	—	19.0% (19/100)	Agent-4 攻击链生成失败 → 弱输入 → 保守拒绝
PR #26	Agent-6 判定规则放宽	38.0% (38/100)	检出翻倍; 分层 43.6% / 37.1% / 10.0%
PR #27	Agent-2 候选风险放宽	无告警样本 0/6 → 3/6	由验证层过滤候选
PR #28	Agent-6 先分析后判定	35.0% (CONFIRMED) / 49.0% (识别)	双口径报告
控制实验	禁用 NVD 数据库	7/10 = 启用时 7/10	排除 NVD 接入为检出断崖根因
OPT-P2 (本工作)	Agent-4 输出契约放宽 + 确定性兜底链	10 样本单轮 5/10 → 6/10	消除断链; 兜底 0 token
OPT-TOKEN (本工作)	Agent-5/6 上游输入压缩	10 样本 token 769,612 → 650,140 (−15.5%)	仅压缩 prompt, 不影响结果落盘
OPT-P1/P3 (本工作)	确定性升级 (高危 + Agent-3 CONFIRMED)	本数据集未触发	WEAK 高危多对应 REFINED, 保持 CONFIRMED 严格语义

4.6 与其他系统报告值的数据对账

表 10 将 HOS-LS 实测值与 2026 年相关系统论文报告值并列，并逐行标注可比性。核心原则：**不同评测集、不同模型、不同口径的数字不做直接优劣判定**——同数据同 API 的受控复测列入 §4.7 待补 (W3)。对账显示：2026 年头部系统的强数字依赖不同评测设定 (PrimeVul 配对样本 / AIXCC 场景) 且存在口径可复现性问题 (§5.1)，而 HOS-LS 在 RepoPairBench 上以公开口径报告双指标；与同模型裸 LLM 的同台对比 (表 8) 才是方法增量的直接证据。

表 10 与其他系统报告值的数据对账

系统	评测集	报告值	与 HOS-LS 的可比性
---	---	---	---
AEGIS [26]	PrimeVul 435 对配对样本	122 对 (pair-wise @k=2, 单次运行无统计检验)	函数级 C/C++ 基准; 同协议复测待补 (W3)
FuzzingBrain V2 [25]	AIXCC AFC (40 漏洞 / 12 项目)	现场 28/63 ≈ 44%; 赛后回测 90%	场景不同 (fuzzing 动态 vs 静态); 口径问题见 §5.1
DREA [27]	RepoPairBench 100 pairs	自消融提 11–16 点 Pair 分; token offload 93%	同评测集; 未公开逐项检出率, 无法直接对账
PrimeVul [7]	BigVul 等 (函数级)	code LM 严格去重后 F1 3.09%	函数级 code LM, 任务设定不同
HOS-LS (全量)	RepoPairBench 100 pairs	静态召回 76.0%; AI 双口径 35.0% / 49.0%	本工作实测, 全部口径公开
HOS-LS (10 样本同台)	RepoPairBench 10 pairs	识别口径 8/10、误报 3/10 (vs 裸 LLM 9/10、9/10)	本工作实测, 同模型受控

4.7 待补实验与口径声明

已完成: RepoPairBench 全量 (静态层 + AI 层双口径)、分层与子集分析、上下文增益实验、机制优化历程、与裸 LLM 的同台受控对比 (10 样本)、OPT 轮受控子集与 token 计量 (n=21 截断子集 CONFIRMED 47.6%; 同 10 样本 token −15.5%)。**待补** (对应工作计划 W1–W5, 不预设数值): ① PrimeVul 函数级 + 仓库级协议 (对齐 AEGIS, 数据下载中); ② Semgrep / CodeQL 同协议基线; ③ 与 AEGIS / DREA 的同数据同 API 受控对比 (L1 主战场); ④ 三层消融 (纯规则 / 规则+LLM 无验证闭环 / 完整系统); ⑤ OPT 配置全量 100 重跑与按层按 Agent 成本表 (预算受限, 当前为 n=21 截断子集); ⑥ 多 seed 统计检验 (当前 10 样本为多轮并集、全量为单次运行)。**口径声明:** 全部指标给出分子/分母与判定定义; 样本以 pair_id 可追溯; n=21 截断子集按文件名序截断、与 v3 全量不直接可比 (已标注); 系统自评数据 (92/100 等) 不入正文, 待第三方协议数据替换。

5 讨论

5.1 与对比文章的口径对账

FuzzingBrain V2 声称的零日数量 29 与 41 相互矛盾，90% 检出为赛后回测而现场成绩约 44%，39/41 个零日无法核验；AEGIS 的 FPR 采用逆向口径 ($fp = P-V + P-R$)，且其 README 声称的文件与仓库不符；DREA 的基准样本存在修复落在测试文件的问题。HOS-LS 的应对是：公开全部口径定义、隐藏基线、分子分母与样本 ID；漏洞声称挂 CVE 编号；评测数据与脚本随论文公开。口径透明不是加分项，而是系统默认能力。

5.2 混合方法的代价与局限

hybrid 并非免费：规则库需持续维护；规则误报会污染 Search Agent 的 Top-K 候选，可能误导 LLM 注意力；规则引擎、RAG、图数据库、验证器等多组件增加部署复杂度。评测层面，函数级设定下 HOS-LS 的检出受样本特性限制（测试文件、跨函数漏洞约占样本 45%）；AI 判定整体保守，部分真实高危被压为 WEAK。同台对比（表 8）进一步揭示代价的另一面：裸 LLM 以 90% 误报换取高召回、工程上不可用；两段式混合精度尚可但召回不足；HOS-LS 的验证链在收敛误报的同时也会把部分真实漏洞（如 0294b9f1 的 XXE）压为 REFINED 而非 CONFIRMED——检出与误报的平衡点由验证链完整度决定，这正是识别口径存在的意义。以上代价与局限均如实报告，不回避——这是与 2026 年同类论文（多死于口径问题）的差异化立场。

5.3 威胁模型

评测集污染：RepoPairBench 基于公开 CVE 构造，存在训练记忆风险，本文通过逐样本归因（表 6）与双口径报告缓解。模型随机性：temperature 0.1 → 0.3 实验显示个别样本检出波动（如 00c73b6e 检出 2→1→0），10 样本对比因此采用多轮运行取并集的口径，全量结果需补充多 seed 统计检验。单次运行：表 4 全量结果为单次运行，方差与显著性检验列入待补工作。

5.4 模型泛化性与单一模型局限

本文所有 AI 层实验均基于单一模型 DeepSeek-v4-flash[35]，实验设计保证了机制优化（PR #26/#27/#28）的贡献是在恒定模型能力下的独立变量，但这也带来模型泛化性 (generalizability) 的局限：本文结论仅在 DeepSeek-v4-flash 上成立，未进行多模型验证，模型无关性不在本文声称范围内。该局限如实披露，不回避。

6 结论与展望

HOS-LS 以「静态规则 → Search Agent → LLM 深度分析 → 确定性验证」四阶段混合架构，将公开口径、可核验声称、确定性验证做成系统默认能力。RepoPairBench 实测：静态层召回 76.0%，AI 层双口径 35.0% / 49.0%，漏检归因于函数级样本特性而非系统缺陷；与裸 LLM 的 10 样本同台对比显示，识别口径 8/10（裸 LLM 9/10）而误报由 9/10 降至 3/10；上下文注入实验证实源码完整性是函数级检测的关键变量；机制优化历程证明工程贡献可独立于模型贡献。下一步：完成仓库级评测、与 AEGIS/DREA 的同数据同 API 受控对比、三层消融与成本量化，形成完整可投稿版本。

参考文献 (35 个引用点，全部可核验；完整 BibTeX 见 references.bib)

[1] Yamaguchi et al. Modeling and Discovering Vulnerabilities with Code Property Graphs. IEEE S&P 2014.
[2] Serebryany. OSS-Fuzz: Google's Continuous Fuzzing Service for Open Source Software. USENIX Security 2017.
[3] Avgustinov et al. QL: Object-Oriented Queries on Relational Data (CodeQL). ECOOP 2016.
[4] Li et al. VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. NDSS 2018.
[5] Zhou et al. Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via GNNs. NeurIPS 2019.
[6] Feng et al. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. EMNLP 2020 Findings.
[7] Ding et al. Vulnerability Detection with Code Language Models: How Far Are We? (PrimeVul). ICSE 2025.
[8] Deng et al. PentestGPT: Evaluating and Harnessing LLMs for Automated Penetration Testing. USENIX Security 2024.
[9] Ullah et al. LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?). IEEE S&P 2024.
[10] Xiong & Zhang. Sifting the Noise: A Comparative Study of LLM Agents in Vulnerability False Positive Filtering. ISSTA 2026.
[11] Ma et al. Combining Fine-Tuning and LLM-Based Agents for Intuitive Smart Contract Auditing (iAudit). ICSE 2025.
[12] Wei et al. Advanced Smart Contract Vulnerability Detection via LLM-Powered Multi-Agent Systems. IEEE TSE 2025.
[13] Yang et al. SWE-Agent: Agent-Computer Interfaces Enable Automated Software Engineering. ICLR 2024.
[14] Sheng et al. LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights. ACM CSUR 2025.
[15] Zhou et al. Large Language Model for Vulnerability Detection and Repair: Literature Review and the Road Ahead. ACM TOSEM 2024.

- [16] Croft et al. IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. arXiv:2405.17238.
- [17] Zhang et al. Vul-RAG: Enhancing LLM-Based Vulnerability Detection via Knowledge-Level RAG. arXiv:2406.11147.
- [18] Lekssays et al. LLMxCPG: Context-Aware Vulnerability Detection Through Code Property Graph-Guided LLMs. USENIX Security 2025.
- [19] Sun et al. LLM4Vuln: A Unified Evaluation Framework for Decoupling and Enhancing LLMs' Vulnerability Reasoning. arXiv:2401.16185.
- [20] Widyasari et al. Let the Trial Begin: A Mock-Court Approach to Vulnerability Detection Using LLM-Based Agents. arXiv:2505.10961.
- [21] Zhou et al. Comparison of Static Application Security Testing Tools and Large Language Models for Repo-Level Vulnerability Detection. arXiv:2407.16235.
- [22] Johnson et al. Why Don't Software Developers Use Static Analysis Tools to Find Bugs? ICSE 2013; Habib & Pradel. How Many of All Bugs Do We Find? ASE 2018. (组合引)
- [23] Li et al. Everything You Wanted to Know About LLM-Based Vulnerability Detection But Were Afraid to Ask. arXiv:2504.13474.
- [24] Risse, Liu & Böhme. Top Score on the Wrong Exam: On Benchmarking in Machine Learning for Vulnerability Detection. arXiv:2408.12986.
- [25] Sheng et al. FuzzingBrain V2: A Multi-Agent LLM System for Automated Vulnerability Discovery and Reproduction. arXiv:2605.21779.
- [26] Fang et al. AEGIS: From Clues to Verdicts — Graph-Guided Deep Vulnerability Reasoning. arXiv:2603.20637.
- [27] Sun & Meng. DREA: Decoupled Reasoning and Exploration Agents for Repository-Level Vulnerability Detection. arXiv:2607.13439.
- [28] Wang et al. ReposVul: A Repository-Level High-Quality Vulnerability Dataset. ICSE 2024.
- [29] Semgrep Inc. Semgrep: Lightweight Static Analysis for Many Languages (工具文档, 实验基线) .
- [30] Zibaeirad et al. AutoTrace: From Patches to Triggers via Agentic Interprocedural Patching and Verification. arXiv:2607.12058.
- [31] Hou et al. Revelio: Cost-Efficient Agentic Memory Safety Vulnerability Discovery. arXiv:2606.22263.
- [32] Yun et al. CLEAR: Causal Context-Based Agentic Reasoning for Vulnerability Detection. arXiv:2608.03134.
- [33] Li et al. Graph Is the Verifier: Agentic Reinforcement Learning for Vulnerability Detection (VulAgentRL). arXiv:2607.26656.
- [34] Wu & Nita-Rotaru. ALIBI: Adaptive Agentic Attacks on LLM-Based Vulnerability Detectors. arXiv:2607.24964.
- [35] DeepSeek-AI. DeepSeek-V4 (deepseek-v4-flash 为实验默认模型) . arXiv:2606.19348.

口径说明：本文所有指标均给出分子/分母与判定定义；漏洞声称挂 CVE/样本 ID；预印本引用注明 arXiv 状态；系统自评数据（92/100 等）不入正文，待第三方协议数据替换。